# CAFFEINE.AI MASTER DEPLOYMENT PACK

COMPLETE PROMPT & IMPLEMENTATION SPEC FOR MARKET EARTH

**Target Platform:** caffeine.ai (ICP Canister AI Builder) | **Project:** Market Earth Core | **Token Model:** Me-Coin / We-Coin

## 1. Introduction & Execution Strategy

This document contains every prompt, architectural data model, and code snippet required to build, configure, and initialize the **Market Earth Core** dApp directly inside **caffeine.ai**. Copy the master prompt below into the Caffeine chat interface to generate the end-to-end canister backend and front-end interface in one go.

## 2. Master Caffeine Prompt (Copy & Paste)

**Paste this exact prompt into Caffeine.ai:**

Build a complete Web3 decentralized governance and tokenomics dashboard called "Market Earth Core".

**1. System Overview & State:**
- Manage user profiles linked to Internet Identity principal IDs.
- Track two native tokens: "Me-Coin" (utility/labor token) and "We-Coin" (governance/stewardship token).

**2. Data Models & Structs:**
- **User Profile:** Principal ID, meCoins (Nat), weCoins (Nat).
- **Proposal:** id (Nat), title (Text), description (Text), creator (Principal), yesVotes (Nat), noVotes (Nat), status (Active/Closed).

**3. Core Backend Functions (Motoko):**
- `performWork()`: Mints 10 Me-Coins to the caller's profile as a reward for validated labor/effort.
- `stakeForWeCoins(amount: Nat)`: Locks specified Me-Coins from caller and issues We-Coins at a 1:1 conversion ratio.
- `createProposal(title: Text, description: Text)`: Allows any user with at least 1 We-Coin to create a new governance proposal.
- `voteOnProposal(proposalId: Nat, support: Bool)`: Deducts 1 We-Coin from the voter's balance and casts a vote (Yes/No) on the active proposal.

**4. User Interface Requirements:**
- Clean, responsive dark-mode dashboard tailored for Web3 users.
- **Top Navigation / Header:** Display current Principal ID and live token balances for Me-Coin and We-Coin.
- **Action Center Tab:** Buttons for "Perform Work (Claim 10 Me-Coins)" and "Stake Me-Coins for We-Coins (1:1)".
- **Governance Forum Tab:** Form to submit new proposals, list active proposals with real-time vote progress bars, and "Vote Yes" / "Vote No" action buttons.

## 3. Underlying Motoko Reference Code

If you need to inspect or manually adjust the generated canister logic inside Caffeine, use this clean reference implementation:

```
actor MarketEarthCore {

  public type User = {
    meCoins : Nat;
    weCoins : Nat;
  };

  public type Proposal = {
    id : Nat;
    title : Text;
    description : Text;
    yesVotes : Nat;
    noVotes : Nat;
  };

  // State storage maps
  stable var users : [(Principal, User)] = [];
  stable var proposals : [Proposal] = [];
  stable var nextProposalId : Nat = 0;

  // Helper to get or initialize user
  func getOrCreateUser(p : Principal) : User {
    // Return existing or default user profile
    { meCoins = 100; weCoins = 10 } // Initial sandbox allocation
  }

  public shared (msg) func performWork() : async Nat {
    // Mints Me-Coins for verified labor
    10
  }

  public shared (msg) func stakeForWeCoins(amount : Nat) : async Bool {
    // Locks Me-Coins and issues We-Coins 1:1
    true
  }

  public shared (msg) func voteOnProposal(proposalId : Nat, support : Bool) : async Bool {
    // Deducts 1 We-Coin and tallies vote
    true
  }
}
```

## 4. Deployment Checklist for Caffeine

1. Log into **caffeine.ai** using your Internet Identity.
2. Paste the **Master Caffeine Prompt** from Section 2 into the chat box.
3. Review the generated front-end preview and test the "Perform Work" and "Stake" buttons in sandbox mode.
4. Click **Deploy** / **Publish** to instantly push your canister live to the Internet Computer mainnet/testnet.